

Week 2 Hands-On Lab: Securing an AI-Powered API

Covers: Authentication, Authorization, RBAC/ABAC, MFA, Encryption, Data Privacy, Audit Logs, Secure APIs

PART 1: Code Lab – Fix the Vulnerable API

Below is a simplified Node.js/Express API for an **AI HR assistant** (same scenario style we discussed). It currently has **3 intentional vulnerabilities** modeled directly on real incidents we discussed (McDonald's Olivia BOLA breach, weak admin credentials, missing audit logs).

Setup

```
mkdir hr-bot-lab && cd hr-bot-lab
npm init -y
npm install express jsonwebtoken
```

server.js (VULNERABLE VERSION – your starting point)

```
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());

const SECRET = "supersecret123"; //  issue 1: hint - where should this

// Fake employee database
const employees = {
  "101": { name: "Asha", department: "Engineering", salary: 90000, manager: "102"},
  "102": { name: "Ravi", department: "Sales", salary: 75000, manager: "201"},
};

// Fake user accounts
```

```

const users = {
  "asha@company.com": { password: "123456", id: "101", role: "employee" }
  "admin@company.com": { password: "admin123", id: "999", role: "admin" }
};

// LOGIN – issues an auth token
app.post('/login', (req, res) => {
  const { email, password } = req.body;
  const user = users[email];
  if (!user || user.password !== password) {
    return res.status(401).json({ error: "invalid credentials" });
  }
  const token = jwt.sign({ id: user.id, role: user.role }, SECRET, { exp: 1000000000 });
  res.json({ token });
});

// AI ASSISTANT ENDPOINT – "show me employee info"
// 🚩 issue 3: no ownership check, no audit log
app.get('/employee/:id', (req, res) => {
  const employee = employees[req.params.id];
  if (!employee) return res.status(404).json({ error: "not found" });
  res.json(employee); // returns EVERYTHING including salary, to ANYONE w
});

app.listen(3000, () => console.log('Vulnerable HR bot API running on :3000'))

```

Your Task – find and fix these 3 issues:

🚩 Issue 1 – Hardcoded secret

The JWT signing secret is hardcoded in source code.

Fix it: Move it to an environment variable (`process.env.JWT_SECRET`). Bonus: explain in one sentence why this connects to the Encryption topic – what does this NOT protect against, even after fixing it?

🚩 Issue 2 – Weak credentials, plaintext passwords

Passwords are stored in plaintext and one is "123456" – the exact password used in the real McDonald's Olivia breach's exposed admin account.

Fix it: Hash passwords with bcrypt before storing/comparing. Enforce a minimum password policy.

🚩 Issue 3 – BOLA (the big one)

GET /employee/:id returns any employee's full record to any authenticated user – no check that the requester owns that record or has a role permitted to view others' records. This is the exact vulnerability class from the McDonald's breach (64M records exposed) and the Meta chatbot account-recovery case.

Fix it – implement this logic:

```
IF requester.id == requested.id → allow (own record)
ELSE IF requester.role == "manager" AND requested.manager ==
requester.id → allow (own report)
ELSE IF requester.role == "admin" → allow
ELSE → deny (403)
```

Bonus: also strip the `salary` field unless the requester is HR/admin – this fixes **property-level authorization**, not just object-level.

🚩 Stretch goal – Add audit logging

Add a middleware that logs every request as:

```
{ "actor": "...", "on_behalf_of": "...", "action": "...", "outcome": "..."
```

PART 2: Breach Case Study – McDonald's "Olivia" (June 2025)

Re-read the case: BOLA vulnerability + exposed admin account (password "123456") exposed 64 million applicants' data, including private chat transcripts.

Answer these yourself, then we'll discuss:

1. Which of the 3 vulnerabilities you just fixed in Part 1 maps directly to this real breach?
 2. The report said "*this wasn't advanced AI going off the rails.*" Given everything we've covered – do you agree? Why?
 3. If you were the security engineer reviewing this system **before** launch, which single control (from Week 2) would you have prioritized checking first, and why?
 4. The admin account was a "test account" that was never deactivated. Which of the 4 A's of IAM does this failure belong to?
-

PART 3: Scenario Decisions – Secure or Not?

For each, decide **Secure**  / **Not Secure**  and say *which specific control* is missing or present.

1. An AI agent uses its own long-lived service API key to query a customer database on behalf of whichever user is chatting with it.
 2. A RAG chatbot retrieves all matching documents from a vector database, then the system prompt says: *"Do not reveal documents outside the user's department."*
 3. A company gives a new AI browser-extension integration "Allow All" scope during setup, planning to narrow it later.
 4. An audit log entry reads: `{"actor": "AI-Agent", "action": "record_deleted", "outcome": "success"}`
 5. A banking chatbot drafts a fund transfer but requires the customer to explicitly confirm before the transfer API is called.
 6. An MFA implementation uses push notifications with no limit on how many can be sent per hour.
-

Answer Key (no peeking until you've tried!)

- ▶ Part 1 hints
 - ▶ Part 3 answers
-

Next step: Once you've worked through Part 1's code, share what you changed and I'll review it like a code review — pointing out anything still missed, the way a security engineer would.